

Computational Simulation of Two-Dimensional Brownian Motion: Diffusion Coefficient and Boltzmann Constant Extraction

Matthew David Carmel Lukacs
College of Science, Australian National University and
u8507410

(Dated: Semester 1, 2026; PHYS2020)

Two-dimensional Brownian motion was simulated in Python using a multi-particle random walk model grounded in Einstein’s diffusion theory. The simulation was developed iteratively through four stages, from an initial lattice walk to a physics-based continuous model with ensemble mean squared displacement (MSD) analysis. Physical constants were chosen to match the conditions of the associated PHYS2013 laboratory experiment⁵. The diffusion coefficient D was extracted via linear regression on the ensemble MSD and used to estimate Boltzmann’s constant k_B via the Stokes–Einstein relation, replicating Perrin’s 1909 experimental approach. Across four independent runs ($T = 298\text{ K}$, $\eta = 8.9 \times 10^{-4}\text{ Pa}\cdot\text{s}$, $a = 0.5\text{ }\mu\text{m}$), the mean fitted diffusion coefficient was $\bar{D}_{\text{fit}} = (4.87 \pm 0.41) \times 10^{-13}\text{ m}^2/\text{s}$, agreeing with the Stokes–Einstein prediction $D_{\text{theo}} = 4.905 \times 10^{-13}\text{ m}^2/\text{s}$ to within 0.8%. The corresponding estimate of Boltzmann’s constant is $\bar{k}_{B,\text{fit}} = (1.37 \pm 0.12) \times 10^{-23}\text{ J/K}$, within 0.8% of the accepted value. An interactive particle box simulation was additionally developed in Pygame demonstrating diffusion as an emergent phenomenon.

I. INTRODUCTION

When a microscopic particle is suspended in a fluid, it undergoes continuous, irregular motion driven by thermal collisions with the surrounding fluid molecules. This phenomenon was documented by Robert Brown in 1828¹ and given a quantitative theoretical framework by Einstein in 1905², who showed that particle displacements follow a Gaussian probability distribution. Perrin subsequently verified this experimentally and extracted Boltzmann’s constant from measured diffusion coefficients, earning the Nobel Prize in Physics in 1926³. Brownian motion provided some of the earliest direct evidence for the atomic nature of matter.

This project computationally replicates Perrin’s approach: simulating the Brownian motion of a 500 nm bead in water, extracting D from the ensemble MSD, and recovering k_B via the Stokes–Einstein relation. The physical constants $T = 298\text{ K}$ and $\eta = 8.9 \times 10^{-4}\text{ Pa}\cdot\text{s}$ match the conditions of the associated PHYS2013 laboratory experiment⁵, allowing direct comparison between the simulated and experimentally measured Boltzmann constant. The simulation was developed iteratively through four stages; an additional interactive Pygame particle box demonstrates diffusion as an emergent microscopic phenomenon.

II. THEORY

Einstein showed² that the probability distribution for the displacement x of a suspended particle along one axis at time t is:

$$f(x, t) = \frac{1}{\sqrt{4\pi Dt}} \exp\left(-\frac{x^2}{4Dt}\right), \quad (1)$$

which is a Gaussian $\mathcal{N}(0, \sigma^2)$ with $\sigma^2 = 2Dt$. For two-dimensional motion in the xy -plane, x and y dis-

placements are independent by symmetry, giving a mean squared displacement⁴:

$$\langle r^2(t) \rangle = \langle x^2 \rangle + \langle y^2 \rangle = 4Dt. \quad (2)$$

The diffusion coefficient D is given by the Stokes–Einstein relation²:

$$D = \frac{k_B T}{6\pi\eta a}, \quad (3)$$

where T is the absolute temperature, η is the dynamic viscosity, and a is the particle radius. Rearranging gives the working equation for k_B :

$$k_B = \frac{6\pi\eta a D}{T}. \quad (4)$$

It follows from Eq. (1) that individual steps along each axis are distributed as $\mathcal{N}(0, \sigma)$ with $\sigma = \sqrt{2D\Delta t}$, which is used to set the simulation step sizes in Section IV.

III. SIMULATION DEVELOPMENT

The simulation was developed iteratively through four stages.

A. Stage 1: Lattice walk

The initial implementation modelled a single particle on a discrete lattice, stepping one unit in a randomly chosen cardinal direction at each timestep. Figure 1 shows a representative trajectory. While this captures the qualitative appearance of Brownian motion, the lattice constraint is unphysical and prevents quantitative comparison with Eq. (1).

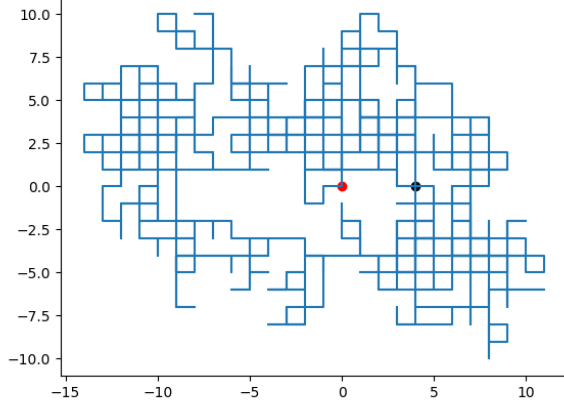


FIG. 1. Stage 1: lattice random walk. Steps are constrained to the four cardinal directions, producing the square-grid appearance. This stage established the basic animation loop but uses no physical constants.

B. Stage 2: Continuous interpolated walk

The lattice constraint was removed by replacing cardinal steps with continuous Gaussian displacements drawn from $\mathcal{N}(0, 1)$, scaled by a step-size parameter. Linear interpolation between steps was added for visual smoothness, removing the grid artefact and producing trajectories that visually resemble real Brownian motion (Fig. 2). However, step sizes were not yet derived from physical constants, limiting quantitative analysis. An animation bug was also identified at this stage: the `FuncAnimation frames` parameter had been set to $n = 5000$ (the number of raw steps), while the interpolated arrays had only $n/k = 500$ entries. This caused an `IndexError` at runtime once the frame index exceeded the array length. The fix was to set `frames=len(x2)`, matching the animation length to the interpolated data.

C. Stage 3: Multi-particle ensemble with MSD

The simulation was extended to an ensemble of $N = 30$ particles. The ensemble MSD (Eq. 5) was computed at each timestep and plotted alongside the theoretical prediction from Eq. (2). A linear regression was applied to extract D_{fit} .

D. Stage 4: Physics-grounded steps and analysis

Step sizes were grounded in physical constants by setting $\sigma = \sqrt{2D_{\text{theo}}\Delta t}$ per axis, where D_{theo} is computed from Eq. (3). A live animation, terminal summary with uncertainties, and post-animation analysis figure were added at this stage.

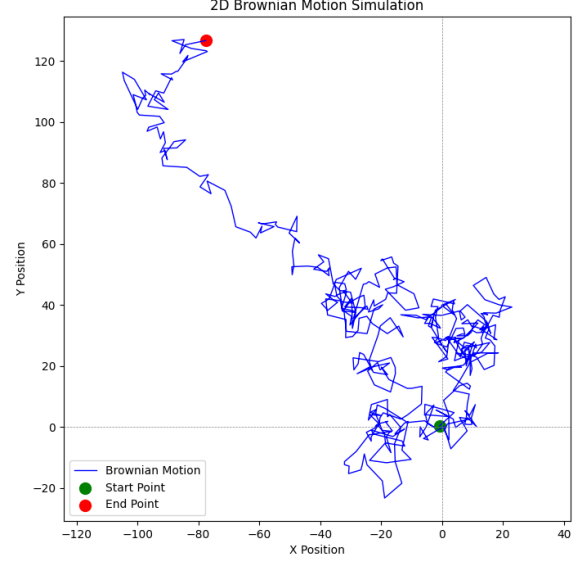


FIG. 2. Stage 2: continuous interpolated walk. Replacing cardinal steps with Gaussian displacements and adding linear interpolation removes the lattice artefact, producing a realistic-looking trajectory. Step sizes are not yet physically motivated.

An inconsistency was identified during development: the initial implementation used $T = 300$ K with $\eta = 1.00 \times 10^{-3}$ Pa·s (corresponding to water at $\approx 20^\circ\text{C}$), inconsistent since $300\text{ K} \approx 27^\circ\text{C}$. This was corrected to $T = 298$ K with $\eta = 8.9 \times 10^{-4}$ Pa·s, matching the PHYS2013 laboratory conditions⁵.

E. Implementation challenges

Several bugs and design issues were identified and resolved across development. Beyond the animation index error at Stage 2, a sign error in the display scale factor (10^{-6} instead of 10^6) caused positions to be displayed in nanometres rather than microns, compressing all trajectories to an invisible point. The T/η inconsistency described above was a subtler error: both values compiled and ran without error, but were physically inconsistent. These are representative of the kinds of silent errors common in scientific computing, where incorrect results are produced without any runtime indication of failure.

For the particle box simulation, an initial implementation of the mean squared displacement tracked the large particle's absolute displacement from its starting position. In a bounded box, this quantity saturates once the particle has explored the available space, driving the fitted slope — and therefore D_{sim} — to zero. The fix was to replace absolute displacement with a windowed approach: measuring $\langle r^2(\tau) \rangle$ over a fixed lag τ , which re-

mains well-defined regardless of box boundaries. A separate issue was that the Stokes drag coefficient, without a scale factor, decayed the large particle’s velocity to near zero between collisions, suppressing diffusion entirely. Introducing a 0.1 scale factor restored physically reasonable behaviour while preserving the qualitative effect of the viscosity slider.

F. Extension: Interactive particle box simulation

The particle box simulation was developed in three tiers of increasing physical fidelity, chosen deliberately to separate the mechanics from the physics. Pygame was used in preference to Matplotlib because the box simulation requires a real-time physics loop advancing state each frame; Matplotlib’s `FuncAnimation` precomputes frames and has pacing limitations under collision-detection load.

The *basic tier* established the core mechanics: one large and $N_{\text{small}} = 20$ uniform-speed small particles, elastic hard-sphere collisions (conserving momentum and kinetic energy), and wall bounces. The *middle tier* introduced physics-based speeds ($v \propto \sqrt{T}$ from Maxwell–Boltzmann, with the large particle slower by $\sqrt{m_{\text{small}}/m_{\text{large}}}$), temperature and viscosity sliders, Stokes drag on the large particle, a trail, and a live windowed MSD panel. The *upgraded tier* added a particle size slider updating r_{large} and $m_{\text{large}} \propto r^2$ in real time, real-time D_{sim} extraction via $\langle r^2(\tau) \rangle / (4\tau)$, and a $D \cdot r$ display for qualitative assessment of the Stokes–Einstein scaling.

IV. METHODS

The simulation parameters are listed in Table I. At each timestep Δt , the displacement of each of the $N = 30$ particles along each axis is drawn independently from $\mathcal{N}(0, \sigma)$ with $\sigma = \sqrt{2D_{\text{theo}}\Delta t}$, following from Eq. (1). Positions are the cumulative sum of these steps. The ensemble MSD is:

$$\langle r^2(t) \rangle = \frac{1}{N} \sum_{i=1}^N [(x_i(t) - x_i(0))^2 + (y_i(t) - y_i(0))^2]. \quad (5)$$

A linear regression (`scipy.stats.linregress`) is applied to MSD versus time, giving slope $m = 4D$ and $D_{\text{fit}} = m/4$. The 1σ uncertainty is propagated from the regression standard error: $\sigma_D = \sigma_{\text{slope}}/4$. Boltzmann’s constant is estimated from Eq. (4). Each of the four reported runs used an independent random seed, so run-to-run variation reflects genuine stochastic scatter.

TABLE I. Simulation parameters.

Parameter	Value
Temperature T	298 K
Viscosity η	8.9×10^{-4} Pa·s
Particle radius a	$0.5 \mu\text{m}$
k_B (NIST ⁶)	1.380649×10^{-23} J/K
D_{theo} (Stokes–Einstein)	4.905×10^{-13} m ² /s
Particles N	30
Timesteps n	800
Δt	0.05 s
Step std σ per axis	$\sqrt{2D_{\text{theo}}\Delta t}$

V. RESULTS

Figure 3 shows the Stage 4 output for a representative unseeded run ($D_{\text{fit}} = 4.85 \times 10^{-13}$ m²/s, $R^2 = 0.987$); no run was discarded or rerun to produce this result. Table II summarises results across four independent runs. Individual percentage errors range from 2.0% to 11.3%, with a mean error of 6.4%. The mean D_{fit} and $k_{B,\text{fit}}$ agree with their accepted values to within 0.8%.

TABLE II. Results across four independent simulation runs. $D_{\text{theo}} = 4.905 \times 10^{-13}$ m²/s; accepted $k_B = 1.381 \times 10^{-23}$ J/K. Uncertainties are 1σ from regression standard error.

Run	D_{fit} (10^{-13} m ² /s)	Err. (%)	R^2	$k_{B,\text{fit}}$ (10^{-23} J/K)
1	4.689 ± 0.027	4.4	0.979	1.320
2	4.516 ± 0.019	7.9	0.987	1.271
3	4.806 ± 0.016	2.0	0.991	1.353
4	5.460 ± 0.038	11.3	0.963	1.537
Mean	4.87 ± 0.41	6.4	0.980	1.37 ± 0.12

Figure 4 shows the post-simulation analysis for a representative run. MSD residuals are small and random at early times but increase in scatter at late times. The displacement step distribution matches the expected Gaussian closely across the full range of step sizes.

VI. DISCUSSION

A. Diffusion coefficient and Boltzmann constant

The mean $\bar{D}_{\text{fit}} = (4.87 \pm 0.41) \times 10^{-13}$ m²/s agrees with the Stokes–Einstein prediction to within 0.8%, and $\bar{k}_{B,\text{fit}} = (1.37 \pm 0.12) \times 10^{-23}$ J/K agrees with the NIST value⁶ to within 0.8%. Individual run errors of 2–11% reflect genuine stochastic variation: with only $N = 30$ particles and $n = 800$ steps, each run represents a different random ensemble realisation. The positive correlation between R^2 and percentage error across runs suggests

Brownian Motion Simulation

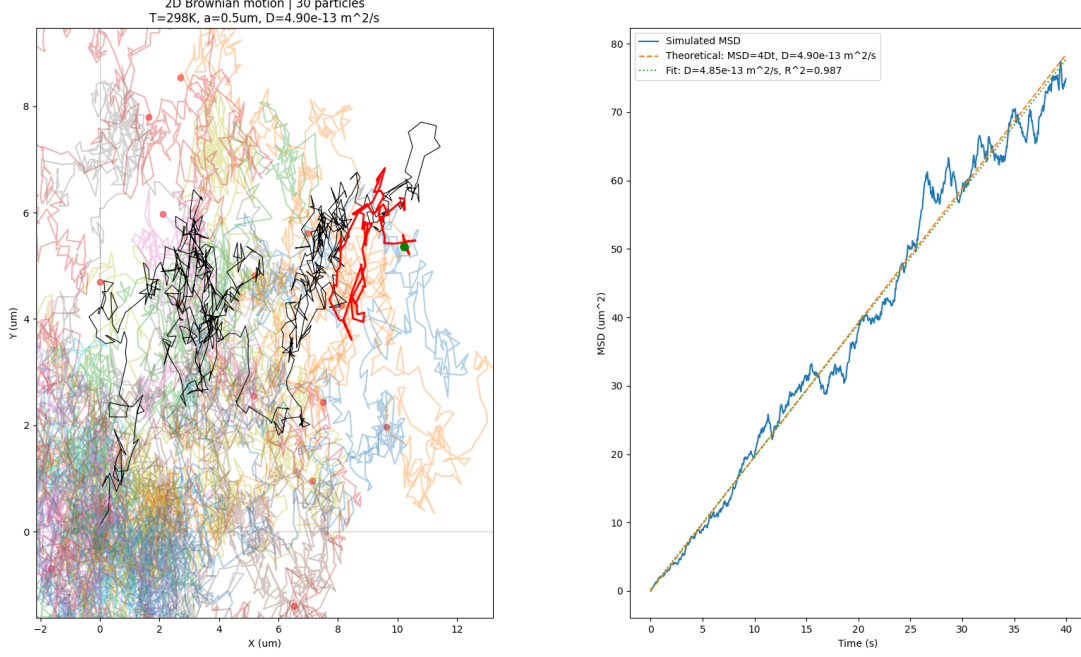


FIG. 3. Stage 4 simulation output. *Left*: 30-particle random walk ensemble ($T = 298$ K, $a = 0.5$ μm). The highlighted particle (black/red trail) is animated live; green dot marks its starting position. *Right*: Ensemble MSD versus time with theoretical prediction from Eq. (2) (orange dashed) and linear regression fit (green dotted). This run: $D_{\text{fit}} = 4.85 \times 10^{-13} \text{ m}^2/\text{s}$, $R^2 = 0.987$.

Post-simulation Analysis

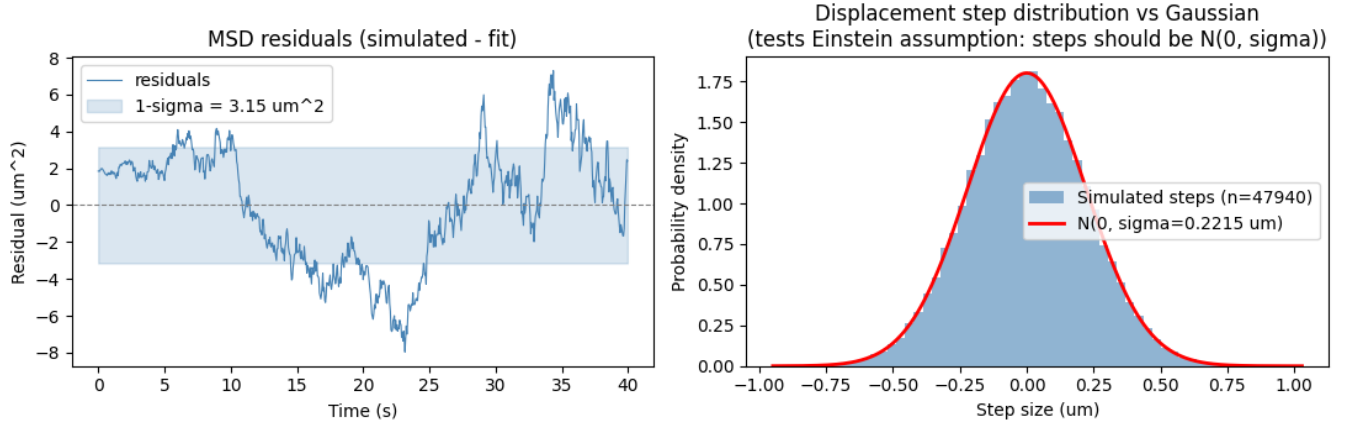


FIG. 4. Post-simulation analysis. *Left*: MSD residuals (simulated MSD minus linear fit) versus time. Shaded band shows $\pm 1\sigma = \pm 3.15 \mu\text{m}^2$. Scatter increases at late times, consistent with finite ensemble sampling effects. *Right*: Displacement step distribution (all axes and particles, $n = 47,940$) versus expected Gaussian $\mathcal{N}(0, \sigma)$ with $\sigma = 0.222 \mu\text{m}$. Agreement is consistent with Einstein's assumption (Eq. 1).

that runs with more linear MSD behaviour tend to yield better D estimates.

B. Comparison with experimental results

In the associated PHYS2013 experiment⁵, k_B was determined from tracked polystyrene microspheres ($a = 0.45 \mu\text{m}$, $T = 299.25$ K). The best experimental result via

MSD analysis gave $k_{B,\text{exp}} = (1.081 \pm 0.043) \times 10^{-23}$ J/K, a 7.0σ underestimate. That report identified untracked z -axis motion, pixel calibration uncertainty, and temperature measurement timing as likely systematic sources, with a sensitivity analysis showing that temperature and viscosity corrections alone could account for at most $\sim 15\%$ of the discrepancy⁵. The simulation is free of these effects by construction (all motion is two-dimensional and particle position is exact), which likely explains why the simulated result agrees more closely with the accepted value despite the shared theoretical framework.

C. Gaussian step distribution

The step histogram in Fig. 4 matches $\mathcal{N}(0, \sigma)$ across the full range, suggesting Einstein’s assumption (Eq. 1) is well-supported and consistent with the step-size scaling $\sigma = \sqrt{2D\Delta t}$. This is the same distribution verified experimentally in the PHYS2013 displacement histograms⁵.

D. Interactive particle box simulation

An interactive simulation of a single large Brownian particle surrounded by smaller fluid molecules was developed in Pygame (Fig. 5). Unlike the random walk model, which pre-computes positions from Gaussian steps, the box simulation advances in real time via explicit elastic collision physics. The large particle’s irregular trajectory (orange trail) emerges naturally from momentum-conserving collisions with the fluid molecules, directly demonstrating the molecular-scale origin of Brownian motion without any prescribed random walk.

Live sliders allow temperature changes (rescaling all molecular speeds by $\sqrt{T_{\text{new}}/T_{\text{prev}}}$, from Maxwell–Boltzmann) and viscosity (applied as Stokes drag: $\Delta v = -6\pi\eta_{\text{sim}}rv\Delta t/m$ per frame on the large particle). A third slider changes the large particle’s radius in real time, updating its mass as $m \propto r^2$. The diffusion coefficient is extracted via $D_{\text{sim}} = \langle r^2(\tau) \rangle / (4\tau)$, where $\langle r^2(\tau) \rangle$ is the mean windowed squared displacement over a fixed lag $\tau = 30$ frames.

The live MSD plot in Fig. 5 illustrates a key contrast with the random walk simulation: the single-particle windowed MSD (red) is highly noisy, fluctuating around the linear fit (green) with large excursions. This is expected — the box simulation tracks only one large particle, whereas `simulation.py` averages the MSD over $N = 30$ particles, suppressing noise by a factor of roughly $\sqrt{N}$. The noisiness of the single-particle MSD is itself physically meaningful, reflecting the stochastic nature of individual Brownian trajectories, and is broadly analogous to the scatter seen in the single-particle tracks of the PHYS2013 experiment⁵.

It is important to note that D_{sim} is in simulation units (px^2/s), not m^2/s , since the box simulation uses a dimensionless viscosity slider and pixel-based velocities rather

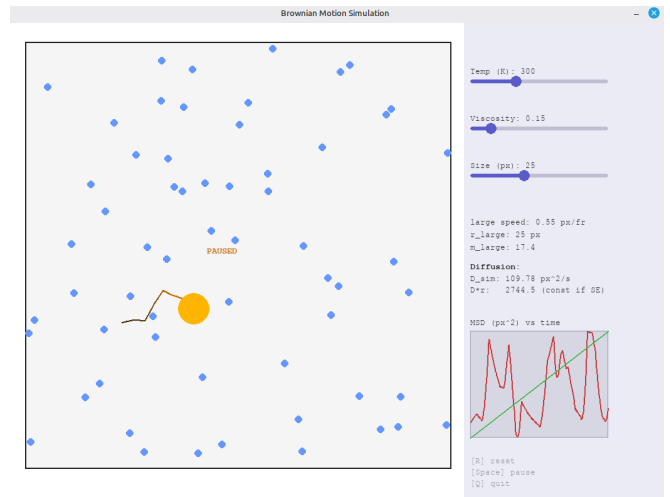


FIG. 5. The upgraded interactive particle box simulation, shown paused. The large orange particle undergoes Brownian motion driven by elastic collisions with the smaller blue fluid molecules. The right panel shows live sliders for temperature, viscosity, and particle size, real-time diffusion statistics, and a live MSD plot (red: windowed $\langle r^2(\tau) \rangle$; green: linear fit). For this run: $D_{\text{sim}} = 109.78 \text{ px}^2/\text{s}$ and $D \cdot r = 2744.5 \text{ px}^3/\text{s}$ at $r = 25 \text{ px}$, $T = 300 \text{ K}$.

than real physical constants. Absolute comparison with the Stokes–Einstein prediction is therefore not meaningful. However, the *scaling* with particle size can be tested qualitatively: Eq. (3) suggests $D \propto 1/r$, equivalently $D \cdot r \approx \text{const}$. For the run shown, $D \cdot r = 2744.5 \text{ px}^3/\text{s}$ at $r = 25 \text{ px}$. A systematic test varying r and checking whether $D \cdot r$ remains approximately constant would more rigorously assess this scaling; this is identified as a natural extension.

VII. CONCLUSION

A Python simulation of two-dimensional Brownian motion was implemented and validated across four independent runs. The mean fitted diffusion coefficient $\bar{D}_{\text{fit}} = (4.87 \pm 0.41) \times 10^{-13} \text{ m}^2/\text{s}$ agrees with the Stokes–Einstein prediction to within 0.8%, and the mean Boltzmann constant estimate $\bar{k}_{B,\text{fit}} = (1.37 \pm 0.12) \times 10^{-23} \text{ J/K}$ agrees with the NIST value⁶ to within 0.8%. The Gaussian step distribution strongly supports Einstein’s 1905 assumption. Comparison with the PHYS2013 result⁵ suggests a key advantage of simulation over experiment: systematic effects from z -axis motion, pixel calibration, and temperature measurement — which likely caused the 7.0σ underestimate in the laboratory — are absent from an idealised numerical model. An interactive Pygame particle box additionally demonstrates Brownian motion as a collision-driven emergent phenomenon.

A. Try it yourself!

The code for both the random walk simulation and the interactive particle box is available

on GitHub at <https://github.com/carmel3141/Brownian-Motion-Simulation-PHYS2020>.

¹ R. Brown, Edinburgh New Philosophical Journal, p. 358 (1828).

² A. Einstein, Annalen der Physik **17**, 549 (1905).

³ J. Perrin, Annales de Chimie et de Physique **18**, 5 (1909).

⁴ F. Reif, *Fundamentals of Statistical and Thermal Physics*, McGraw-Hill, New York (1965), Ch. 15.

⁵ M. D. C. Lukacs, PHYS2013 Laboratory Report: Brownian Motion, Australian National University (2026). Unpub-

lished.

⁶ National Institute of Standards and Technology (NIST), <https://physics.nist.gov/cgi-bin/cuu/Value?k> (accessed 2026).

⁷ Water viscosity data table, <https://chem-casts.com/data-tables/water-viscosity-data-table> (accessed 2026).

⁸ ANU PHYS2020 Laboratory Manual, Brownian Motion Simulation (2026).